

From Demo to Production

Building AI Agents That Survive the Real World

François Noel — Extrait : Introduction + Partie 1 (chapitres 1.1–1.4)

Cet extrait est partagé à titre personnel. Merci de ne pas le redistribuer.

From Demo to Production: Building AI Agents That Survive the Real World

Introduction

J'ai passé 12 mois à construire des agents IA en production.

Pas en démo.

Pas dans un notebook Jupyter.

Pas dans un environnement sandbox où tout fonctionne parfaitement.

En production réelle. Avec des utilisateurs réels. Des conséquences réelles.

Et j'ai vu des choses que les benchmarks ne montrent jamais.

J'ai vu un agent RH valider un contrat avec une clause de non-concurrence illégale — parce qu'on lui avait donné l'autonomie de valider sans lui donner les garde-fous pour le faire correctement. J'ai vu un agent de support client inventer une procédure de remboursement qui n'existait pas, promettre 200€ à un client, et créer un incident financier qu'il a fallu gérer manuellement pendant deux semaines. J'ai vu des agents internes — prometteurs en démo, unanimement approuvés en démo — devenir inutilisables en production, abandonnés par les équipes au bout de six semaines.

Ces échecs ont un point commun. Ce n'est pas GPT-4 qui était en cause. Ce n'est pas Claude. Ce n'est pas Llama.

C'est l'architecture.

D'où viennent ces leçons

J'écris ce guide parce que j'ai construit des agents qui ont tenu en production — et d'autres qui n't tenu qu'en démo. Les leçons viennent de là : revise.studio (context engineering sur fiction longue), Hotelix (workflow hôtelier où l'agent était la mauvaise réponse), Ralph Loop (agent de développement autonome qui re-dérivait tout sans mémoire externe), et des missions de consulting dans des contextes que je n'identifierai pas.

Ce ne sont pas des leçons abstraites. Ce sont des erreurs que j'ai commises, des corrections que j'ai apportées, et des patterns que j'ai observés se répéter d'un projet à l'autre.

Le problème qu'on ne voit pas

L'industrie a un biais. Quand un agent échoue, on cherche d'abord du côté du modèle. On change le prompt. On passe à un modèle plus puissant. On ajoute du few-shot. On optimise le chain-of-thought. Et parfois ça aide — mais le plus souvent, le problème est ailleurs.

Le problème, c'est qu'on a construit un agent là où un workflow aurait suffi. Ou qu'on a déployé en autonomie complète un agent qui n'avait pas encore prouvé qu'il pouvait prendre des décisions fiables. Ou qu'on a traité la fenêtre de contexte comme un buffer infini et qu'on s'est retrouvé avec un agent qui perd le fil sur des tâches longues. Ou qu'on n'a jamais pensé à la mémoire entre les sessions. Ou que la boucle ReAct s'effondre à la quatrième itération parce qu'il n'y a aucun mécanisme de planification. Ou

qu'on a ajouté l'évaluation après coup — c'est-à-dire jamais vraiment.

Ces six erreurs, je les ai vues se répéter. Pas une fois. Pas deux fois. À chaque projet, dans des équipes différentes, avec des contextes différents, elles réapparaissaient.

Ce guide est construit autour de ces six erreurs — et de ce qui vient après : la sécurité, la conformité, la production, les outils actuels, et les preuves terrain.

Ce que ce guide n'est pas

Ce n'est pas un tutoriel sur les LLMs. Si tu ne sais pas ce qu'est un context window, un pattern ReAct, ou du retrieval augmenté (RAG), commence par les bases et reviens ici.

Ce n'est pas non plus un comparatif de frameworks. LangChain, LlamaIndex, CrewAI, LangGraph, OpenAI Agents SDK — ils ont tous leur place, aucun ne règle les problèmes architecturaux qu'on va aborder. Ce guide est **framework-agnostic** : les patterns qu'il décrit s'implémentent dans n'importe quelle stack. Comprendre *pourquoi* un contrat standardisé entre agent et outil est nécessaire vaut plus que de savoir configurer un serveur MCP. Comprendre *pourquoi* les guardrails existent vaut plus que de connaître l'API InputGuardrail. Les standards changent — MCP, A2A, Agents SDK sont apparus en 2024–2025. Les problèmes qu'ils résolvent existaient avant eux, et existeront après leurs successeurs.

Et ce n'est pas une liste de best practices génériques du type "écris de bons prompts". Ces conseils sont vrais mais insuffisants. On va aller plus loin.

Les six erreurs que ce guide corrige

Erreur 1 : Construire un agent là où un workflow aurait suffi. L'agent n'est pas le marteau universel. Il y a des cas où la complexité d'un agent est injustifiée — et où un workflow simple, déterministe, serait plus fiable, moins coûteux, et plus maintenable.

Erreur 2 : Chercher l'autonomie immédiate. L'autonomie complète est une destination, pas un point de départ. Un agent qu'on déploie sans filets de sécurité est un agent qui va faire des erreurs coûteuses.

Erreur 3 : Traiter le contexte comme un paramètre illimité. "J'ai 200K tokens, je peux tout mettre dedans." Non. La fenêtre de contexte est une ressource limitée qui se gère activement.

Erreur 4 : Ignorer l'amnésie fondamentale des LLMs. Un LLM est stateless. Chaque requête repart de zéro. Sans architecture mémoire explicite, ton agent perd le fil sur les tâches longues et ne capitalise jamais sur son expérience passée.

Erreur 5 : Utiliser des boucles ReAct naïves. Le pattern ReAct fonctionne bien en démo. En production, sur des tâches complexes, il s'effondre. Les erreurs s'accumulent. Les boucles divergent.

Erreur 6 : Ajouter l'évaluation après coup. L'évaluation qu'on prévoit d'ajouter "quand le système sera stable" ne se fait jamais. Sans évaluation dès le début, on ne détecte pas le drift comportemental, et on ne peut pas améliorer ce qu'on ne mesure pas.

Ce que les Parties 7 à 11 ajoutent

Les six premières parties posent les fondations architecturales. Mais elles ne couvrent pas tout ce qu'un agent en production doit affronter.

Partie 7 — Sécurité : le threat model spécifique aux agents. Prompt injection directe et indirecte, exfiltration de données, sandboxing des outils, moindre privilège. Fondé sur l'OWASP LLM Top 10 (2025).

Partie 8 — Conformité et gouvernance : où ton agent tombe dans le cadre réglementaire (EU AI Act, NIST AI RMF). Ce n'est pas de la paperasse — c'est une contrainte architecturale qui détermine ce que tu peux construire et comment.

Partie 9 — Production operations : un agent est un système distribué. Idempotence, retries, saga pattern, RBAC, isolation multi-tenant, SLO, rollback, runbooks. Ce qui manque pour passer de "ça fonctionne" à "ça tient en production".

Partie 10 — État de l'art 2025–2026 : MCP, A2A, OpenAI Agents SDK, OpenTelemetry, sandboxing (E2B, Docker). Ces standards sont des implémentations des principes des parties précédentes — pas des substituts.

Partie 11 — Études de cas : cinq projets réels (contexte gouvernemental, outils internes, revise.studio, Hotelix, Ralph Loop). Pas des exemples fabriqués. Des cas avec leurs hésitations, leurs erreurs et leurs corrections.

Comment lire ce guide

Chaque partie est autonome. Tu peux les lire dans l'ordre ou sauter directement à l'erreur qui correspond à ton problème du moment.

Chaque partie se termine avec un livrable concret : une grille de décision, un modèle de déploiement, un framework de gestion du contexte, une architecture mémoire, un pattern de planification, un système d'évaluation, un threat model, une grille de gouvernance, une checklist de production-readiness, ou une matrice de choix d'outillage.

Les exemples viennent du terrain. Les noms et détails sont changés là où la confidentialité l'exige, mais les situations sont réelles.

Le code est simplifié pour la clarté. L'objectif n'est pas de te donner du code production-ready — c'est d'illustrer les concepts avec des patterns précis.

Un agent n'est pas un prompt intelligent.

C'est un système.

Et construire un système qui survit au monde réel — pas seulement à la démo — ça s'apprend.

Commençons.

L'agent n'est pas le marteau universel

Il y a un biais que j'ai observé chez presque tous les développeurs qui découvrent les agents — moi y compris.

On vient de finir son premier agent en démo. Ça marche. Le modèle raisonne, il appelle des outils, il produit un résultat. On est impressionné. Et alors un réflexe s'installe : *"Et si j'utilisais ça pour...?"*

Pour automatiser les rapports. Pour trier les emails. Pour valider les dépenses. Pour planifier les réunions. Pour tout.

C'est le réflexe du marteau. Quand tu as un marteau, tout ressemble à un clou.

Le problème, c'est que les agents ne sont pas des marteaux. Ce sont des outils spécialisés, avec un coût d'utilisation réel — et ce coût ne se voit pas en démo.

Exemple concret :

Une équipe RH m'a contacté pour construire un "agent de gestion des demandes de congé". L'idée semblait raisonnable : les employés soumettent des demandes, l'agent vérifie les soldes, applique les règles de l'entreprise, valide ou refuse.

On a passé six semaines à construire l'agent. Gestion du contexte, gestion des exceptions, intégration avec le SIRH, gating sur les cas limites. Beau travail.

Trois mois après le déploiement, le taux d'erreur était de 12%. Un employé sur huit recevait une réponse incorrecte — mauvais solde calculé, règle mal appliquée, cas limite mal géré.

Le problème ? La gestion des congés dans cette entreprise suivait des règles fixes, documentées, prévisibles. Soixante-dix pour cent des demandes suivaient exactement le même schéma. Les trente pour cent restants avaient des exceptions connues et cataloguées.

Ce cas n'avait pas besoin d'un agent. Il avait besoin d'un workflow avec quelques conditions et une interface propre. On aurait pu le construire en une semaine. Il aurait eu un taux d'erreur proche de zéro.

Ce que j'ai mal évalué, c'est la complexité réelle de la tâche. J'ai confondu *complexité d'implémentation* et *complexité de décision*.

La complexité d'implémentation était réelle : intégrer plusieurs systèmes, gérer des cas d'exception, respecter des règles métier. C'est du travail.

Mais la complexité de décision était faible : les règles étaient connues, les cas couverts, les exceptions documentées. Il n'y avait pas de jugement contextuel à exercer. Aucune adaptation dynamique nécessaire. Aucune information manquante à inférer.

Un agent excelle quand la décision est complexe — quand il faut raisonner sur des informations partielles, adapter la réponse au contexte, naviguer dans l'ambiguïté. Pas quand il faut appliquer des règles déterministes sur des données structurées.

Utiliser un agent dans le second cas, c'est payer le prix de la flexibilité sans en tirer le bénéfice.

Ce prix est plus élevé qu'on ne le croit.

Le développement d'un agent prend en général trois à cinq fois plus longtemps qu'un workflow équivalent. Pas parce que le code est plus complexe — mais parce que le comportement non

déterministe d'un LLM exige des tests d'un autre ordre. On teste des chemins, pas des valeurs. On anticipe des dérives, pas des bugs.

Ensuite vient le monitoring. Un workflow échoue clairement : erreur levée, log produit, alerte déclenchée. Un agent échoue silencieusement. Il répond toujours quelque chose. C'est ce quelque chose qui peut être faux, incomplet, ou légèrement hors-sujet — sans que le système ne le détecte.

Et il y a la maintenance. Les modèles évoluent. Ce qui fonctionnait avec GPT-4-turbo se comporte différemment avec la version suivante. Un workflow est stable dans le temps. Un agent, non.

```
# 'L Surcouche agent sur une tâche déterministe
def process_leave_request(request: dict) -> dict:
    context = build_context(request, hr_policies, employee_data)
    response = llm.generate(context, "Validate this leave request")
    return parse_response(response) # Non déterministe, fragile

# ' Workflow simple et fiable
def process_leave_request(request: dict) -> dict:
    balance = get_employee_balance(request["employee_id"])
    if balance < request["days"]:
        return {"status": "refused", "reason": "insufficient_balance"}
    if conflicts_with_team_schedule(request):
        return {"status": "pending", "reason": "schedule_conflict"}
    return {"status": "approved"}
```

La version workflow est plus courte, plus lisible, plus testable. Elle ne nécessite pas de LLM. Et elle ne produira jamais 12% d'erreurs.

La vraie question à se poser avant de construire un agent, ce n'est pas *"Est-ce que je pourrais faire ça avec un agent ?"* — la réponse est presque toujours oui. La vraie question, c'est *"Est-ce que j'ai besoin d'un agent pour faire ça ?"*

Ces deux questions n'ont pas les mêmes réponses.

Dans les sections suivantes, on va tracer la frontière précise entre agent, workflow et automation. Et on va construire les critères qui permettent de choisir — sans se laisser piéger par l'enthousiasme.

[SCHÉMA 1.1] Workflow déterministe vs agent adaptatif — quand chaque approche s'applique

Trois mots, trois réalités

On utilise ces trois termes de façon interchangeable.

Agent, workflow, automation. Dans les conversations d'équipe, dans les pitches, dans les README des projets. Comme si le mot importait peu — comme si c'était juste une question de branding.

Ce n'est pas juste une question de vocabulaire. Ce sont trois architectures fondamentalement différentes, avec des propriétés, des coûts et des limites radicalement différentes. Confondre les trois, c'est comme confondre un script shell, une API REST et un système distribué. Techniquement, les trois "font des trucs automatiquement". Mais on n'en choisit pas un par hasard.

Clarifions une fois pour toutes.

L'automation : la règle, rien que la règle

L'automation, c'est simple. Presque bête. Et c'est exactement pourquoi elle est efficace.

Une automation exécute une action déterministe en réponse à un déclencheur prédéfini. Pas de raisonnement. Pas d'adaptation. Pas d'ambiguïté tolérée.

Un nouvel utilisateur crée un compte !' envoyer un email de bienvenue. Une facture dépasse 10 000 € !' alerter le responsable financier. Un ticket reste ouvert depuis 48 heures !' escalader au niveau 2.

```
# ' Automation : déterministe, lisible, testable
def on_invoice_received(invoice: Invoice):
    if invoice.amount > 10_000:
        notify_finance_manager(invoice)
    else:
        auto_approve(invoice)
```

Le comportement est entièrement prévisible. Il n'y a pas de "peut-être". Il n'y a pas d'"ça dépend du contexte". La règle s'applique, l'action s'exécute.

C'est une force, pas une faiblesse. Une automation ne peut pas déraiper. Elle ne peut pas interpréter une règle différemment selon son humeur. Elle ne rate pas parce qu'un modèle a évolué.

Là où ça pose problème, c'est quand on utilise l'automation pour des cas qui nécessitent une lecture contextuelle. La règle est trop rigide. Le cas réel déborde toujours légèrement du cadre prévu. Et l'automation ne sait pas que faire avec ce débordement — elle l'ignore, ou elle plante.

Le workflow : l'orchestration sans surprise

Le workflow va un cran plus loin. C'est une séquence d'étapes prédéfinie, potentiellement avec des branches conditionnelles, mais dont le périmètre est connu à l'avance.

La différence avec l'automation ? La complexité de l'orchestration. Un workflow coordonne plusieurs étapes, plusieurs systèmes, plusieurs acteurs — mais le chemin global est balisé. On sait d'où on part, où on va, et quelles bifurcations sont possibles.

Exemple concret :

Un workflow de validation de dépenses en entreprise. L'employé soumet une note de frais. Le workflow vérifie si la dépense entre dans le budget prévu. Si oui, il la transfère au manager pour approbation. Si le

manager approuve, elle part en comptabilité. Si le montant dépasse 5 000 €, elle va d'abord au directeur financier. Si quelque chose est manquant — justificatif absent, catégorie non renseignée — le workflow renvoie à l'employé avec un message précis.

```
# ' Workflow : orchestration prévisible avec branches connues
def process_expense(expense: Expense) -> WorkflowResult:
    if not expense.has_receipt():
        return request_receipt(expense)

    if expense.amount > 5_000:
        approval = get_cfo_approval(expense)
    else:
        approval = get_manager_approval(expense)

    if approval.granted:
        return send_to_accounting(expense)
    else:
        return notify_rejection(expense, approval.reason)
```

Toutes les branches sont explicites. Tout le monde peut lire ce code et comprendre exactement ce qui se passe. Les tests sont directs — on couvre chaque branche, on valide chaque sortie.

Ce workflow aurait-il besoin d'un agent ? Non. Les règles sont connues, les cas sont couverts, les exceptions sont documentées. Comme l'exemple des congés qu'on a vu en 1.1 — complexité d'implémentation réelle, complexité de décision faible.

Là où le workflow atteint ses limites, c'est quand les exceptions deviennent imprévisibles. Quand les règles ne suffisent plus parce que la réalité déborde le modèle. Quand il faut lire entre les lignes.

L'agent : le raisonnement sous incertitude

Un agent, c'est différent en nature. Pas seulement en complexité.

Un agent observe son environnement, raisonne sur ce qu'il perçoit, décide quelle action prendre, l'exécute, observe le résultat, et recommence. C'est la boucle Reason !' Act que le pattern ReAct formalise. Ce qui distingue un agent d'un workflow, c'est cette capacité d'adaptation à l'imprévu.

Un workflow suit un chemin. Un agent choisit son chemin à chaque étape.

[SCHÉMA 1.1] Comparaison : automation (trigger !' action), workflow (étapes balisées), agent (boucle Reason !' Act avec adaptation dynamique)

Exemple concret :

Un agent de tri de tickets support. Chaque ticket est différent — formulation vague, problème multifacettes, contexte utilisateur variable. L'agent doit comprendre la demande même quand elle est mal exprimée, identifier la catégorie pertinente parmi des dizaines de possibilités, décider si le ticket nécessite une escalade immédiate ou peut suivre la file standard, et parfois poser une question de clarification avant d'agir.

```
# ' Agent : raisonnement contextuel sur un problème non déterministe
def triage_ticket(ticket: Ticket, tools: ToolSet) -> TriageResult:
    # L'agent observe le contexte
    context = build_context(ticket, tools.get_user_history(ticket.user_id))

    # Il raisonne (ReAct : Reason !' Act !' Observe !' Repeat)
    while not is_decision_final(context):
        reasoning = llm.reason(context)
        action = llm.decide_action(reasoning, tools.available_actions())
        result = tools.execute(action)
```



```
context = update_context(context, action, result)

return context.final_decision
```

Ce que fait l'agent ici n'aurait pas pu être scripté à l'avance. La diversité des tickets, la formulation imprévisible, la nécessité de croiser plusieurs informations avant de trancher — c'est précisément là qu'un workflow déterministe aurait craqué.

Choisir, c'est éliminer

En pratique, la question n'est pas "agent ou workflow ?" mais "quel est le niveau de décision contextuelle requis ?"

Si la réponse est "aucun — les règles couvrent tous les cas", c'est une automation.

Si la réponse est "les étapes sont connues, mais il faut orchestrer plusieurs systèmes et gérer des branches", c'est un workflow.

Si la réponse est "les cas débordent toujours légèrement du cadre, et il faut raisonner sur l'ambiguïté", c'est — peut-être — un agent.

[TABLEAU 1.1] Grille de sélection : Automation / Workflow / Agent selon critères (prévisibilité des règles, adaptabilité requise, coût d'erreur, complexité d'orchestration)

Le "peut-être" est important. Parce qu'un agent coûte. En développement, en monitoring, en maintenance. On n'y vient que quand les deux autres options ont montré leurs limites.

Ce qu'on ne fait pas : on ne commence pas par l'agent parce que c'est "plus impressionnant". On commence par l'automation, on monte vers le workflow si nécessaire, et on n'arrive à l'agent que si le problème l'exige vraiment. L'architecture suit le problème — pas l'inverse.

Dans la section suivante, on traduit ça en signaux concrets. Pas de la théorie — des critères observables, directement applicables à ton prochain projet.

Les signaux qui ne trompent pas

Dans la section précédente, on a clarifié les définitions : automation, workflow, agent. Trois architectures, trois niveaux de complexité de décision.

Mais une définition ne suffit pas pour décider. Ce qu'il faut, ce sont des signaux concrets — des indicateurs observables dès la phase de design, avant d'avoir écrit une seule ligne de code.

J'en ai identifié quatre. Chacun, pris seul, devrait te faire hésiter. Deux ou plus ensemble, c'est un signal d'alarme.

Signal 1 : le workflow est linéaire et prévisible

La première question à poser sur n'importe quelle tâche : est-ce que le chemin est toujours le même ?

Pas "est-ce que c'est compliqué ?" Pas "est-ce que ça implique plusieurs systèmes ?" La question est : les étapes se suivent-elles dans un ordre prédéfini, sans bifurcations inattendues ?

Si tu peux dessiner le workflow sur un tableau blanc — avec des boîtes et des flèches — et que ce schéma couvre 95% des cas réels, tu n'as pas besoin d'un agent. Tu as besoin d'un workflow.

Un agent excelle dans l'imprévisibilité. C'est son domaine. Il raisonne sur des situations qu'on n'a pas anticipées, navigue dans l'ambiguïté, adapte son chemin à mesure que l'information arrive. Si la tâche ne requiert rien de tout ça, payer le prix de cette flexibilité est un gaspillage pur.

J'ai vu une équipe produit construire un "agent de gestion de releases" pendant trois mois. Déployer en staging, vérifier les métriques, notifier Slack, créer le tag Git, mettre à jour Jira. Cinq étapes. Toujours dans le même ordre. Jamais d'exception.

C'est un pipeline de CI/CD. Pas un agent.

Signal 2 : les décisions peuvent être scriptées

Deuxième question : si on donnait la liste des règles métier à un stagiaire, pourrait-il prendre les mêmes décisions que l'agent ?

Ce n'est pas une question sur l'intelligence. C'est une question sur la nature des décisions. Si les règles sont explicites, documentées, et couvrent les cas réels — la décision est scriptable. Et ce qui est scriptable n'a pas besoin d'un LLM pour être pris.

```
# 'L Agent sur une décision déterministe
def approve_expense(expense: dict) -> bool:
    context = build_context(expense, policies, history)
    response = llm.generate(
        f"Analyse cette note de frais et dis si elle doit être approuvée: {context}"
    )
    return parse_approval(response)

# ' Règles explicites – lisibles, testables, déterministes
def approve_expense(expense: dict) -> bool:
    if not expense.get("receipt"):
        return False
    if expense["amount"] > 5000:
        return requires_cfo_approval(expense)
    if expense["category"] not in ALLOWED_CATEGORIES:
```

```
return False
return expense["amount"] <= budget_remaining(expense["employee_id"])
```

La version agent coûte plus cher à faire tourner, elle peut produire des résultats inconsistants selon la formulation de la note de frais, et elle est impossible à tester unitairement de façon fiable. La version règles est testable, auditable, et se comporte de façon identique à chaque exécution.

Quand les règles métier sont claires et stables, les encoder explicitement est toujours préférable à les laisser inférer à un modèle.

Signal 3 : aucune adaptation contextuelle n'est nécessaire

Troisième signal : est-ce que la réponse correcte change selon le contexte utilisateur, l'historique, la situation particulière — ou est-ce que la même input donne toujours la même output ?

C'est la question qui distingue l'agent du workflow le plus clairement. Un agent justifie son existence quand deux demandes identiques en surface nécessitent des réponses différentes selon le contexte. Un ticket support qui dit "je ne peux pas me connecter" mérite des réponses différentes selon que l'utilisateur a un compte actif, vient de changer son mot de passe, ou accède depuis un nouveau pays.

Si la tâche n'a pas besoin de lire ce contexte pour décider — si la réponse est la même quelle que soit la situation particulière de l'utilisateur — un workflow suffit.

Signal 4 : le coût de l'erreur est élevé

Ce signal est différent des trois précédents. Il ne dit pas "tu n'as pas besoin d'un agent". Il dit : "si tu utilises un agent ici, sois extrêmement prudent — et probablement, tu ne devrais pas."

Les agents se trompent. Pas tout le temps, pas de façon catastrophique en général — mais ils se trompent. C'est inhérent au non-déterminisme des LLMs. Une reformulation légèrement différente peut produire une décision différente. Un passage de la base de connaissance mal récupéré peut orienter le raisonnement dans la mauvaise direction.

Dans des domaines où l'erreur est réversible et peu coûteuse, c'est acceptable. Dans des domaines critiques, c'est une dette que tu ne veux pas contracter.

Exemple concret :

Un client dans le secteur juridique voulait automatiser la validation de clauses contractuelles. L'idée : un agent lit les contrats soumis par les RH, identifie les clauses problématiques, et valide ou rejette.

L'agent fonctionnait bien sur les cas types. Il identifiait correctement les clauses de non-concurrence abusives, les périodes d'essai illégales, les mentions obligatoires manquantes.

Mais il avait un taux d'erreur de 8% sur les cas atypiques. Pas énorme. Sauf que dans ce contexte, 8% d'erreur sur des contrats de travail — c'est une clause illégale passée, un licenciement mal documenté, une exposition légale pour l'entreprise.

On a changé l'approche. L'agent n'est plus le validateur — il est l'assistant. Il analyse le contrat, produit une liste de points d'attention, et soumet cette liste à un juriste pour validation finale. Le juriste prend la décision. L'agent réduit son temps de travail de 70%.

C'est la bonne architecture pour les domaines critiques : l'agent accélère, l'humain décide.

[SCHÉMA 1.2] Continuum de risque : de l'automatisation complète à la validation humaine obligatoire

Utiliser ces signaux en pratique

Ces quatre signaux ne sont pas des règles absolues. Ils sont des questions à poser avant de commencer.

Quand tu évalues un nouveau use case, passe-les en revue :

Le workflow est-il linéaire ? Les décisions sont-elles scriptables ? L'adaptation contextuelle est-elle nécessaire ? Le coût de l'erreur est-il élevé ?

Si tu réponds "oui, non, non, oui" — tu as probablement un cas pour un workflow robuste avec quelques règles métier bien encodées, pas un agent.

```
# Grille de décision rapide – à adapter à ton contexte
def needs_agent(task: dict) -> bool:
    # Signal 1 : workflow linéaire
    if task["flow_is_linear"] and task["steps_are_fixed"]:
        return False

    # Signal 2 : décisions scriptables
    if task["decisions_are_rule_based"] and task["rules_are_documented"]:
        return False

    # Signal 3 : pas d'adaptation contextuelle
    if not task["requires_contextual_adaptation"]:
        return False

    # Signal 4 : coût d'erreur élevé sans supervision
    if task["error_cost"] == "high" and not task["has_human_in_loop"]:
        return False # pas "pas d'agent", mais "pas sans supervision"

    # Si aucun de ces signaux ne disqualifie l'agent potentiellement justifié
    return True
```

Ce code est délibérément simple. Ce n'est pas une implémentation — c'est une checklist encodée. L'idée, c'est de forcer la conversation sur ces questions avant de se lancer.

La bonne question n'est pas "est-ce qu'on peut faire ça avec un agent ?" On peut presque tout faire avec un agent. La bonne question, c'est "est-ce qu'on *doit* ?"

La réponse, souvent, est non.

Dans la section suivante, on regarde ce que ça coûte vraiment quand on dit oui sans avoir posé ces questions.

Ce que tu vois, et ce que tu ne vois pas

Le premier devis d'un agent est toujours trop optimiste.

Deux semaines de développement, disent-ils. Un sprint et demi. On intègre l'API, on écrit quelques prompts, on déploie. Simple.

Ce n'est pas un mensonge. C'est juste incomplet.

Le problème avec les agents, c'est que leur vrai coût n'apparaît pas dans la PR initiale. Il émerge progressivement, en production, sous forme de petites dettes qui s'accumulent. Chaque erreur inexpliquée, chaque comportement inattendu, chaque mise à jour de modèle qui casse subtilement quelque chose — tout ça a un prix. Et ce prix, on ne l'a pas calculé au départ.

J'ai appris ça à mes dépens. Pas une fois. Plusieurs.

Le développement : la partie visible de l'iceberg

Construire un agent correctement prend en moyenne trois à cinq fois plus longtemps qu'un workflow équivalent. Pas parce que le code est plus complexe. Parce que le comportement non déterministe d'un LLM change fondamentalement ce que "tester" veut dire.

Dans un workflow, on teste des valeurs. `assert result == expected`. Pass ou fail. Simple.

Dans un agent, on teste des comportements. Est-ce que le raisonnement est cohérent sur 50 variantes de la même requête ? Est-ce que l'agent ne se bloque pas quand un outil renvoie une réponse malformée ? Est-ce qu'il maintient le bon fil directeur sur dix étapes consécutives ?

Ces questions n'ont pas de réponse binaire. Elles demandent des jeux de données d'évaluation, des runs répétés, de l'analyse manuelle.

Exemple concret :

Une startup m'a contacté pour déboguer un agent de qualification de leads commerciaux. L'agent interrogeait le CRM, analysait le profil de l'entreprise, et catégorisait les prospects en trois niveaux de priorité.

Ils avaient estimé le développement à deux semaines. Il avait pris six. Pas à cause d'imprévus techniques. Parce qu'ils avaient découvert en cours de route que l'agent produisait des catégorisations inconsistantes sur des profils similaires. Le même lead soumis deux fois avec un libellé légèrement différent recevait parfois une priorité différente.

Corriger ça a demandé de construire un jeu d'évaluation de cent cas, de faire tourner l'agent en boucle, et d'itérer sur les prompts jusqu'à atteindre une cohérence acceptable. Ce n'était pas du développement. C'était du travail éditorial sur le comportement du modèle.

Un workflow de qualification avec des règles explicites aurait pris dix jours. Et il aurait produit exactement le même résultat sur le même input, à chaque fois.

Le monitoring : le coût qui ne s'arrête jamais

Un workflow échoue clairement. Exception levée. Log produit. Alerte déclenchée. L'erreur est visible, localisable, reproductible.

Un agent échoue silencieusement.

Il répond toujours quelque chose. Mais ce quelque chose peut être faux, incomplet, légèrement hors-sujet — sans que le système ne le détecte. Pas de stack trace. Pas d'alerte. Juste une mauvaise décision qui passe à travers.

C'est ce que j'appelle l'**échec silencieux**. Et c'est ce qui rend le monitoring d'un agent radicalement plus coûteux que celui d'un workflow.

Pour monitorer un agent correctement, il faut :

- Logger non seulement les outputs, mais le raisonnement (pourquoi l'agent a pris cette décision)
- Construire des métriques sur la qualité des décisions, pas seulement sur la disponibilité du service
- Faire de l'évaluation en continu sur un échantillon des décisions prises
- Détecter les dérives comportementales avant qu'elles impactent les utilisateurs

```
# 'L Monitoring minimal – ne voit que les crashes
def agent_decide(context: dict) -> dict:
    result = agent.run(context)
    logger.info(f"Agent responded: {result['output']}")
    return result

# ' Monitoring complet – capture les décisions et leur justification
def agent_decide(context: dict) -> dict:
    result = agent.run(context)
    decision_log = {
        "input_hash": hash(str(context)),
        "decision": result["output"],
        "reasoning": result.get("reasoning", ""),
        "confidence": result.get("confidence"),
        "tools_called": result.get("tool_calls", []),
        "timestamp": datetime.now().isoformat(),
    }
    decision_store.append(decision_log)
    metrics.record_decision(decision_log)
    return result
```

Ce logging structuré est indispensable. Sans lui, on ne peut pas détecter qu'un agent dérive, ni comprendre pourquoi il a pris une mauvaise décision trois semaines plus tard.

La maintenance : le coût du temps

Les modèles évoluent. GPT-4-turbo se comporte différemment de GPT-4o. Claude 3 Opus et Claude 3.5 Sonnet ne répondent pas aux mêmes prompts de la même façon. Et quand un provider déploie une nouvelle version — parfois sans te prévenir — ton agent peut subtilement changer de comportement sans que rien ne plante.

C'est ce que j'appelle le **drift de modèle**. Et c'est une source de coût de maintenance qui n'existe pas avec un workflow.

Un workflow qu'on a écrit en 2022 tourne en 2026 exactement pareil. Il n'y a pas de modèle sous-jacent qui évolue. Les règles sont les règles.

Un agent demande des tests de régression après chaque changement de modèle. Des passes de vérification après chaque mise à jour de dépendances. Un suivi de la roadmap des providers pour anticiper les changements.

[TABLEAU 1.2] TCO comparatif : workflow vs agent sur 12 mois

Calculer avant de construire

Voici une estimation réaliste du TCO d'un agent simple sur douze mois, comparé à un workflow équivalent :

```
# Estimation TCO – agent vs workflow (en jours-ingénieur)
tco = {
  "workflow": {
    "développement": 10,    # dev initial
    "tests": 3,            # tests déterministes
    "déploiement": 1,
    "monitoring": 2,       # alertes standard + logs
    "maintenance_12m": 4,  # corrections de bugs, évolutions
    "total": 20,
  },
  "agent": {
    "développement": 25,    # dev + prompt engineering + eval
    "tests": 10,           # tests comportementaux, jeux d'eval
    "déploiement": 3,       # staging, tests de charge
    "monitoring": 15,       # dashboards décisions, analyses drift
    "maintenance_12m": 20,  # drift modèle, évolutions de comportement
    "total": 73,
  }
}

ratio = tco["agent"]["total"] / tco["workflow"]["total"]
# !' 3.65x plus coûteux sur 12 mois
```

Ce ratio de 3,5x à 4x est cohérent avec ce que j'ai observé sur le terrain. Et il ne tient pas compte des coûts d'inférence — les appels LLM coûtent plusieurs ordres de grandeur plus cher que l'exécution d'un workflow Python.

Pour un cas où un workflow suffit, ce surcoût est pur gaspillage. Tu paies le prix de la flexibilité d'un agent sans en avoir besoin.

La question à poser avant de commencer n'est pas "est-ce qu'on peut construire ça comme un agent ?" Elle est : "est-ce que la valeur ajoutée de l'agent justifie un TCO quatre fois plus élevé ?"

Souvent, la réponse est non.

Dans la section suivante, on traduit tout ça en une règle simple — le critère unique qui permet de trancher.